

西南交通大學
Southwest Jiaotong University

《算法分析与设计》

算法效率分析

计算机与人工智能学院
李晓航
xhli_scce@swjtu.edu.cn

Last: 2026-03



一、算法分析框架

- 1.1 算法分析概述
- 1.2 算法复杂性
- 1.3 算法复杂性函数的输入
- 1.4 时间复杂性函数的输出
- 1.5 时间复杂性函数定义
- 1.6 最差、最优和平均时间复杂性
- 1.7 算法分析过程

1.1 算法分析概述

- 算法分析（Algorithm Analysis）：
  - 对算法所需要的两种计算机资源—时间资源和空间资源进行估算。
- 算法分析的目的：
  - 设计算法—设计出复杂性尽可能低的算法。
  - 选择算法—在多种算法中选择其中复杂性最低者。
- 算法分析的结果是算法复杂性。

1.2 算法复杂性

- 算法复杂性（Algorithm Complexity）
  - 指算法执行时所消耗或占用的资源（时间和空间）的量。
  - 一个算法所需资源越多，它的复杂性越高，反之则低。
- 算法复杂性包括
  - 时间复杂性（Time Complexity）：指出了算法的运行时间情况。
  - 空间复杂性（Space Complexity）：指出了算法所需要的额外辅助空间的情况。
- 算法分析包括时间复杂性和空间复杂性分析
  - 由于计算机硬件技术的发展，现在对空间复杂性的考虑较少，而主要关注时间复杂性。
  - 本课程只讨论算法的时间复杂性分析。
- 思考：如何定义时间复杂性函数 y=f(x) ？

1.3 算法复杂性函数的输入

- 显然，算法的时（空）间复杂性与算法的输入规模密切相关
  - 一般情况下，当输入规模显著增加时，算法的运行时间也会增加，算法的时（空）间复杂性会显著增长。
  - 因此，算法复杂性函数的输入参数就是算法的输入规模（通常用 n 表示）。
- 需要注意的是，选择不同的输入规模度量时，算法复杂性的计算结果可能不一致
  - 例：对于两个 n 阶方阵相乘，如果选择 n 作为输入度量，其时间复杂性为n^3（只考虑乘法）；如果选择矩阵元素个数N作为输入度量，则时间复杂性为 $N^{3/2}$。
  - 因此时间复杂性分析时必须确定算法的输入度量。

1.4 时间复杂性函数的输出

- 用算法的绝对运行时间作为时间复杂性函数的输出似乎是可行的
  - 绝对运行时间：算法中所有操作的运行时间之和。
- 绝对运行时间的计算公式：
  - 设算法输入规模为 n，第 i 步操作记为$\boldsymbol{op_i}$，该步骤的执行时间为 $\boldsymbol{C_{op_i}}$，执行次数为$\boldsymbol{C_i(n)}$，则该算法的总执行时间为：

$$T(n) = \sum_i c_{op_i} * C_i(n) \approx c_{op} * C(n)$$

  - 其中 C_{op} 为算法中所占比例最大的操作（称为基本操作）的执行时间；$C(n)$ 为基本操作的执行次数。

时间复杂性函数的输出（续）

- 用绝对时间作为时间复杂性的问题：
  - 不同计算机的运算速度不同，用绝对时间无法公平比较时间复杂性。

机器	运算速度	运行算法	运行时间
天河三号	百亿亿次/秒	插入排序	无法公平比较
个人电脑	十亿次/秒	选择排序	



VS



时间复杂性函数的输出（续）

- 在实际研究中，为了评价算法的优劣，忽略计算机性能对算法运行时间的影响，只要分析和比较算法中各种操作的执行次数 $C_i(n)$ 即可，这样时间复杂性分析就可以独立于机器。
- 因此，时间复杂性函数的输出为算法中各种操作的执行次数之和。

1.5 时间复杂性函数定义

- 算法的时间复杂性函数为：

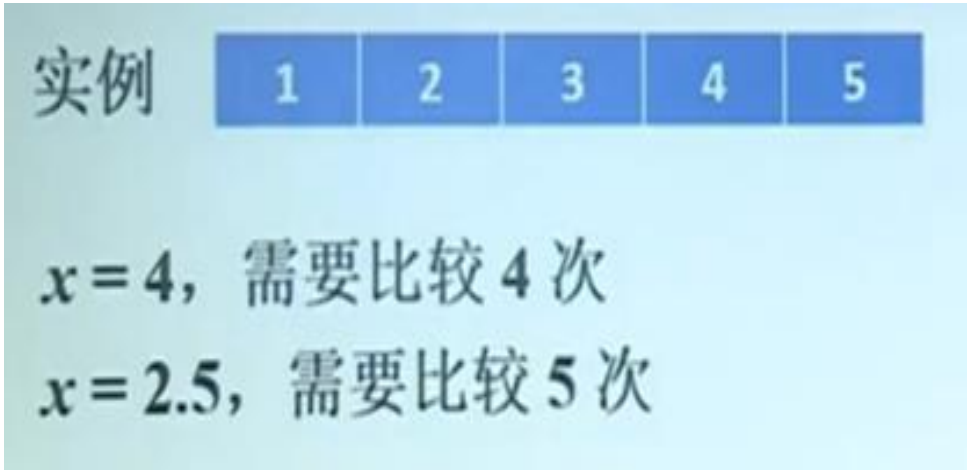
$$T(n) = \sum_{i} C_i(n) \approx C(n)$$

  - 其中为 $C_i(n)$ 为算法中第 i 个步骤的执行次数，C(n) 为基本操作的执行次数。
  - 因此，计算时间复杂性只需要算出算法基本操作的执行次数即可。
- 需要注意的是：大多数算法的时间复杂性不仅取决于输入规模 n，还与每次运行时具体的输入实例有关。

输入实例对时间复杂性的影响

- 例：顺序查找算法（输入规模是数组长度 n）：

```
int seqSearch(a[1..n],  x )
{
    for(int i=1; i<=n; i++)
            if (a[i]==x)  return i;
    return -1;
}
```



显然对于不同输入实例，该算法的时间复杂性是不同的。

其最好和最坏情况为：

$C_{best}(n) = 1$，即最优情况下只需比较 1 次。

$C_{worst}(n) = n$，即最差情况下需比较 n 次。

算法的平均复杂性

- 平均复杂性与算法输入实例的概率分布有关，其定义为：

$$C_{avg}(n) = \sum_i p(i)C(i)$$

  - 其中 p(i) 为第 i 种输入实例的出现概率，C(i) 为该实例的基本操作执行次数。
- 对于顺序查找问题，输入实例共有 2n+1 种情况，分别对应：
  - 查找成功：x=a[1], x=a[2],…, x=a[n] （ n种 ）
  - 查找不成功：x<a[1], a[1]< x<a[2], a[2]< x<a[3],…, x > a[n] （ n+1种 ）

例：求顺序查找算法平均复杂性

- 设查找成功的概率为 p（每种情况的出现概率相同，即 p/n），查找失败的概率为1-p，则：

$$C_{avg}(n) = n*(1-p) + \sum_{i=1}^{n} i * \frac{p}{n}$$

$$= n*(1-p) + \frac{p}{n} * \sum_{i=1}^{n} i$$

$$= n*(1-p) + \frac{p}{n} * \frac{n(n+1)}{2}$$

$$= n*(1-p) + \frac{p*(n+1)}{2}$$

查找失败下的比较次数

i 是查找成功下每种情况的比较次数

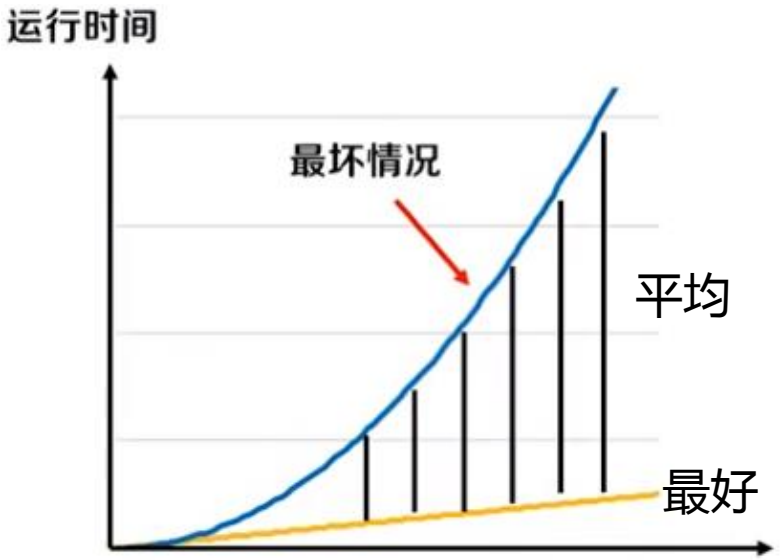
最终结果不仅与输入规模n有关，还与 p 有关

1.6 最差、最优和平均时间复杂性

- 对于受输入实例影响的算法，理论上需要分析最差、最优和平均复杂性。
- 最差复杂性
  - 算法在最坏情况下的运行效率（运行时间最长）。
  - $C_{worst}(n) = \max\{ C(I) \mid size(I)=n \}$
- 平均复杂性
  - 算法在一般或随机情况下的运行效率。
  - $C_{avg}(n) = \sum_I p(I)C(I)$ （ p(I) 是实例 I 出现的概率 ）
- 最优复杂性
  - 算法在最好情况下的运行效率（运行时间最短）。
  - $C_{best}(n) = \min\{ C(I) \mid size(I)=n \}$

最差、最优和平均时间复杂性（续）

输入情况	复杂性	说明
最好情况	最优	不常出现，不具有普遍性
最坏情况	最差	确定算法复杂性的上界，具有一般性
随机情况	平均	需要考虑输入的概率分布，分析和计算难度大



在实际应用中，通常以“最坏”情况作为算法复杂性的代表。



1.7 算法分析过程

- 1、确定输入规模度量参数
  - 如输入数组的元素个数、数据的 bit 位，图的顶点数和边数，等等。
- 2、确定算法基本操作
  - 如加、减、乘、除、比较、置指针、等等。
- 3、确定效率分析目标
  - 最差、平均、最优效率。
  - 重点关注最差效率。
- 4、确定算法复杂度的阶
  - 只需关心算法在输入规模变化时的变化趋势。

例：求最大最小值

- 设计一个尽可能高效的算法，在长度为n的一维数组 a[n] 中查找值最大的元素max和最小的元素min，并分析算法的最好、最坏和平均时间复杂度。

```
void MaxMin(int a[],int n,int &max,int &min)
{
  int i;
  max=min=a[0];
  for(i=1; i<n; i++)
     if(a[i]>max) max=a[i];
     else if(a[i]<min) min=a[i];
}
```

求最大最小值（续）

- 最好情况
  - a 中元素递增排列，元素比较次数为 n-1。
- 最坏情况
  - a 中元素递减排列，元素比较次数为 2*(n-1)。
- 平均情况
  - a 中有一半元素比 max 大， a[i]>max 比较执行 n-1 次，a[i]<min 比较执行 (n-1)/2 次。
  - 因此平均元素比较次数为 3*(n-1)/2 次。

二、算法渐进分析

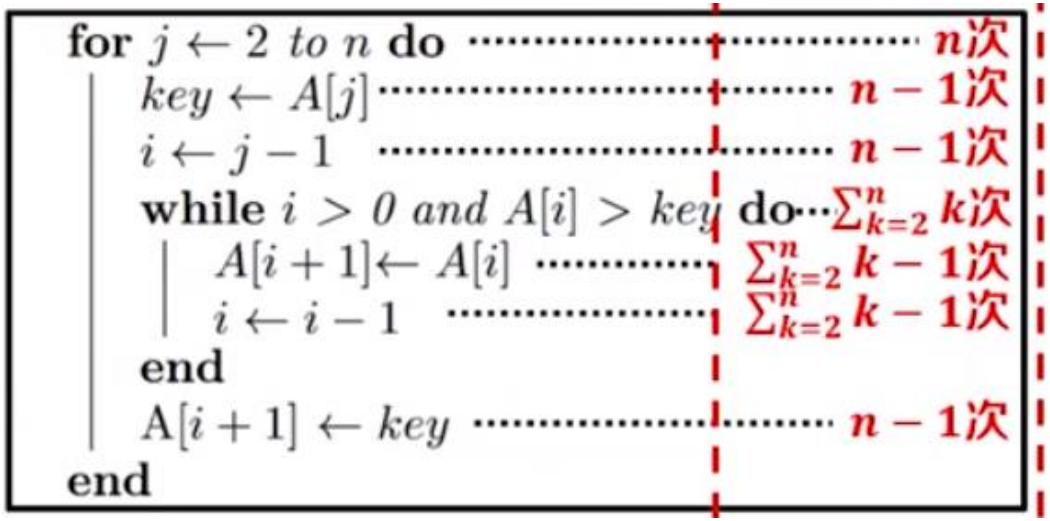
- 2.1 渐进分析的概念
- 2.2 渐进上界（大O）
- 2.3 渐进下界（大Ω）
- 2.4 渐进紧确界（大Θ）
- 2.5 渐进符号的特性
- 2.6 基本复杂度类型
- 2.7 复杂性函数的大类

2.1 渐进分析的概念

- 精确计算和比较算法运行时间比较麻烦，而且当输入规模 n 充分大时，有些看似不同的复杂性函数其实差别不大。
- 渐进分析
  - 分析当问题规模 n 充分大时算法复杂性函数 T(n)的大小关系，类似于极限的思想。
  - 在具体实现渐进分析时通常会忽略复杂性函数 T(n) 的系数和低阶项，仅关注高阶项。
  - 渐进分析的结果用渐进符号表示，主要包括大O、大Ω和大Θ。
    - 注意，这三个符号都代表一类函数的“集合”，具体的复杂性函数 T(n) ∈某个符号。

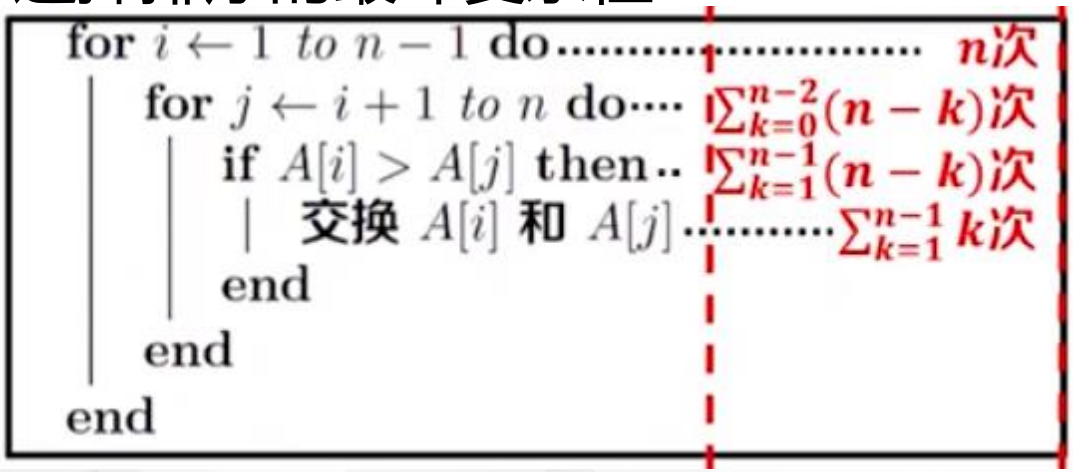
一个复杂性分析的例子

插入排序的最坏复杂性



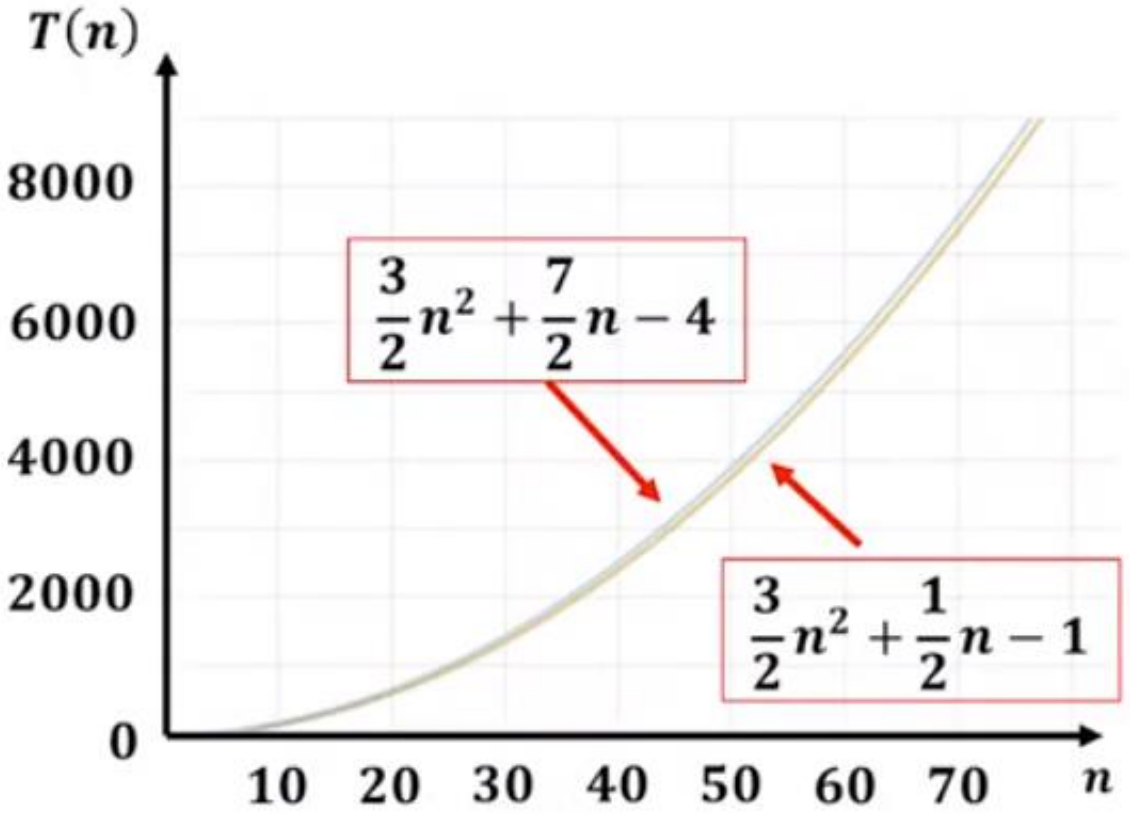
$$T(n)=\frac{3}{2}n^2+\frac{7}{2}n-4$$

选择排序的最坏复杂性



$$T(n)=\frac{3}{2}n^2+\frac{1}{2}n-1$$

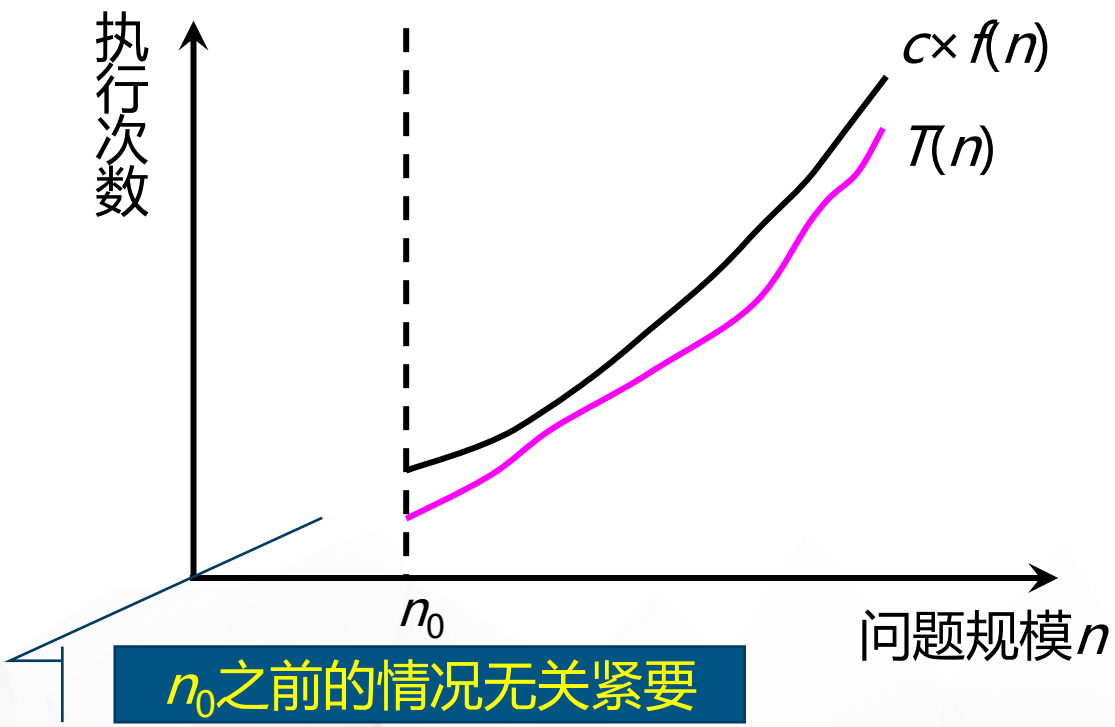
一个复杂性分析的例子（续）



结论：当 n 充分大时，二者相差不大。
原因：两个函数的最高阶（n^2）相同，即这两个函数 $T(n) \in O(n^2)$ 。

2.2 渐进上界

- 记号：大O
- 定义：若存在两个正的常数 c 和 n_0，对于任意 $n \geq n_0$，都有 $T(n) \leq c*f(n)$，则称 f(n) 是 T(n) 的渐进上界，记为：**$T(n) \in O(f(n))$**



注：常数 c 实际消除了 T(n) 中最高项系数的作用，常数 n_0 消除了 T(n) 中低阶项的作用。

另外，大O适用于分析最坏复杂性。

渐进上界（续）

例：证明 $T(n) = 5n^3+3n+12 \in O(n^3)$

证：显然对于 $n \geq 1$，有

$5n^3+3n+12 \leq 5n^3+3\ n^3+12\ n^3=20n^3$，

因此选 $c=20$， $n_0=1$，对于$n \geq n_0$，有$T(n) \leq c \times n^3$ ，

所以 $T(n) \in O(n^3)$。

例：证明 $T(n) = n! \in O(n^n)$

证：显然对于$n \geq 1$，有$n! = n(n-1)(n-2) \ldots 1 \leq n^n$，

因此选 $c=1$，$n_0=1$，对于$n \geq n_0$，有 $T(n) \leq c \times n^n$，

所以 $T(n) \in O(n^n)$。

思考：上界的阶越低，则评估就越？

渐进上界（续）

例：计算 T(n) = $\sum_{i=1}^{n}\frac{1}{i} = O(?)$ （假设 n=2^k）

解：$\sum_{i=1}^{n}\frac{1}{i} = \frac{1}{1}+\frac{1}{2}+\frac{1}{3}+\frac{1}{4}+\frac{1}{5}+\frac{1}{6}+\frac{1}{7}+\frac{1}{8}+\cdots+\frac{1}{n}$

$< \frac{1}{1}+\frac{1}{2}+\frac{1}{2}+\frac{1}{4}+\frac{1}{4}+\frac{1}{4}+\frac{1}{4}+\frac{1}{8}+\cdots+\frac{1}{n}$

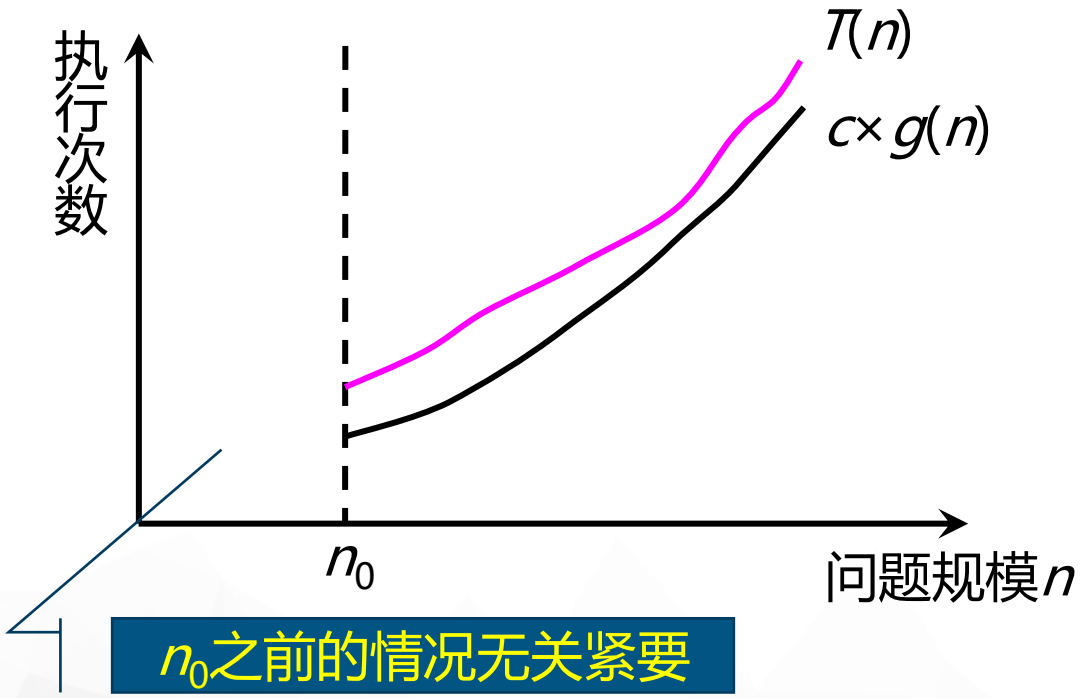
= 1+2*(1/2)+4*(1/4)+8*(1/8)+…+1/n

$= \frac{1}{n}+\sum_{i=1}^{\log n} 1 = \frac{1}{n}+logn = O(logn)$

分析渐进上界通常采用放大法

2.3 渐进下界

- 记号：大Ω
- 定义：若存在两个正常数 c 和 n_0，对于任意 $n \geq n_0$，都有 $T(n) \geq c \times g(n)$，则称 g(n) 是 T(n) 的渐进下界，记为：**$T(n) \in \Omega(g(n))$**



渐进下界（续）

例： 证明 $T(n) = 5n^2 + 8n + 1 \in \Omega(n^2)$

证：显然当n≥1时，有 $5n^2 + 8n + 1 \geq 1 \times n^2$

因此选 c=1，n_0=1，对于n≥n_0，有 $T(n) \geq c \times n^2$

所以 $T(n) \in \Omega(n^2)$

例：计算 $T(n) = \sum_{i=1}^{n} \frac{1}{i} = \Omega(?)$（假设 $n=2^k$）

解： $\sum_{i=1}^{n} \frac{1}{i} = \frac{1}{1} + \frac{1}{2} + \frac{1}{3} + \frac{1}{4} + \frac{1}{5} + \frac{1}{6} + \frac{1}{7} + \frac{1}{8} + \cdots + \frac{1}{n}$

$> \frac{1}{1} + \frac{1}{2} + \frac{1}{4} + \frac{1}{4} + \frac{1}{8} + \frac{1}{8} + \frac{1}{8} + \frac{1}{8} + \cdots + \frac{1}{n}$

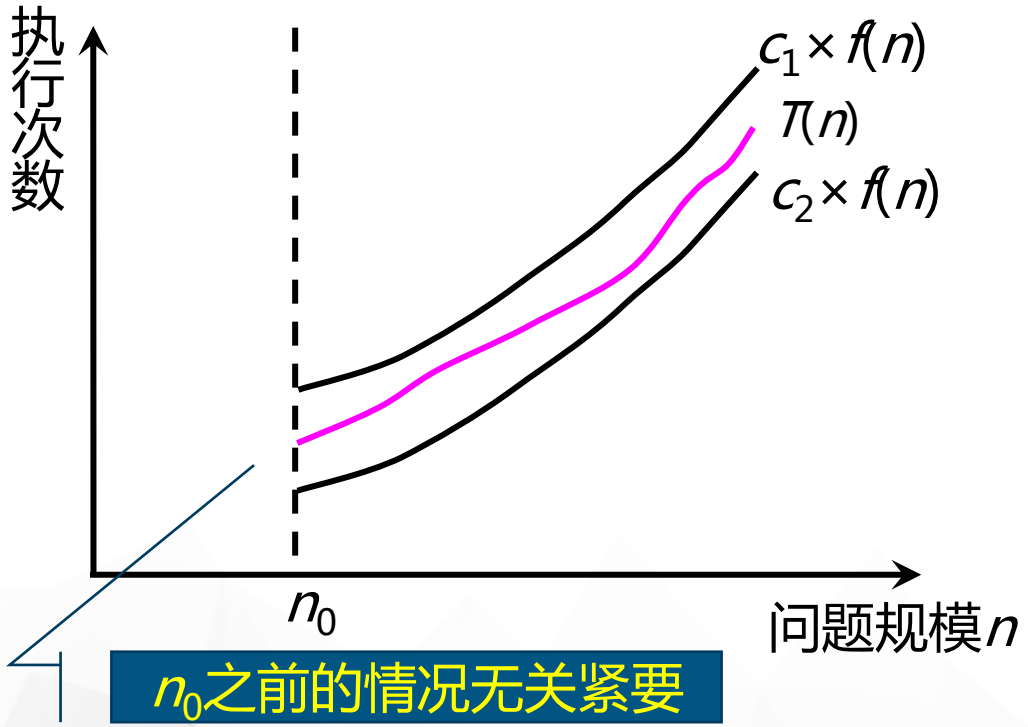
= 1+1/2+2*(1/4)+4*(1/8)+…+(n/2)1/n

$= 1 + \sum_{i=1}^{\log n} \frac{1}{2} = 1 + \frac{1}{2} \log n = \Omega(\log n)$

分析渐进下界通常采用缩小法

2.4 渐进紧确界

- 记号：大Θ
- 定义：若存在三个正常数c_1、c_2和n_0，对于任意$n \geq n_0$，都有$c_1 \times f(n) \geq T(n) \geq c_2 \times f(n)$，则称f(n)是T(n)渐进紧确界，记为：**$T(n) \in \Theta(f(n))$**



渐进紧确界（续）

例：证明 $T(n) = 5n^2 + 8n + 1 \in \Theta(n^2)$

证：当$n \geq 1$时，$5n^2 + 8n + 1 \leq 5n^2 + 8n + n$

$= 5n^2 + 9n \leq 5n^2 + 9n^2 \leq 14\ n^2 = O(n^2)$

当 $n \geq 1$时，$5n^2 + 8n + 1 \geq n^2 = \Omega(n^2)$

因此当 $n \geq 1$ 时，取$c1=14$，$c2=1$，则

$14n^2 \geq 5n^2 + 8n + 1 \geq n^2$

所以 $5n^2 + 8n + 1 \in \Theta(n^2)$

2.5 渐进符号的特性

- 定理1
  - 对于任意两个函数f(n)和g(n)，只有在$f(n)\in O(g(n))$和$f(n)\in \Omega(g(n))$的情况下，$f(n)\in \Theta(g(n))$才成立。
- 定理 2
  - 如果 $f_1(n) \in O(g_1(n))$，$f_2(n) \in O(g_2(n))$，则
    $$f_1(n)+f_2(n) \in O(\max(g_1(n),g_2(n)))$$
  - 该性质对于 Ω 和 Θ 也成立。
- 定理3
  - 如果 $f_1(n) \in O(g_1(n))$，$f_2(n) \in O(g_2(n))$，则
    $$f_1(n)*f_2(n)= O(g_1(n)*g_2(n))$$
  - 该性质对于 Ω 和 Θ 也成立。

渐进符号的特性（续）

- 通过求极限的方法可以计算两个函数变化速度的快慢，从而决定哪一项占主导地位。

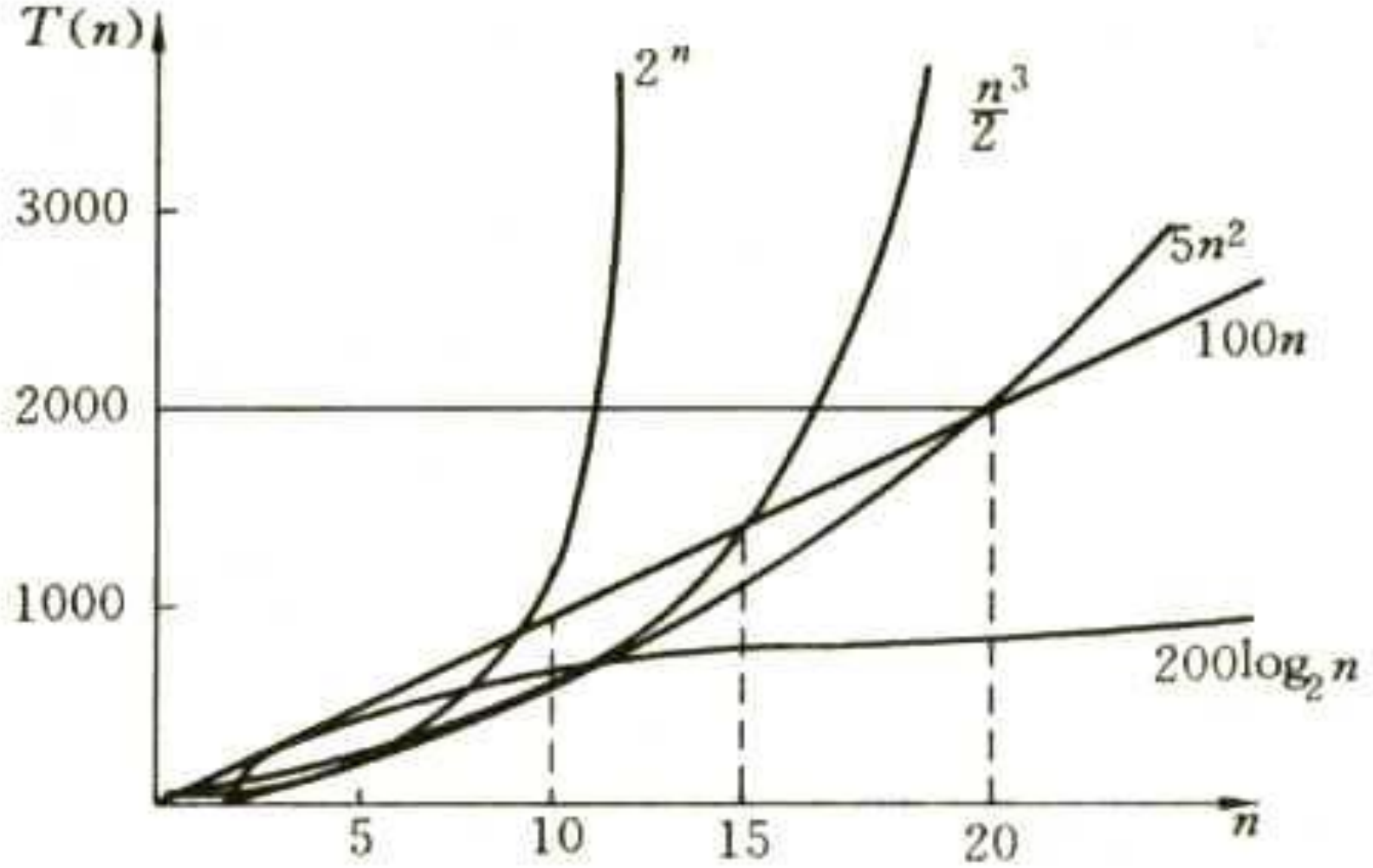
$$\lim_{n\to\infty}\frac{t(n)}{g(n)}=\begin{cases}0 & t(n)\text{的增长次数比}g(n)\text{的小}\\ c>0 & t(n)\text{的增长次数和}g(n)\text{相同}\\ \infty & t(n)\text{的增长次数比}g(n)\text{的大}\end{cases}$$

- 对于第一种情况：t(n) ∈ O(g(n))
- 对于第二种情况：t(n) ∈ Θ(g(n))
- 对于第三种情况： t(n) ∈ Ω(g(n))

2.6 基本复杂度类型

- 算法的基本复杂度类型如下（n是输入规模）
  - 1、常数：复杂度为固定值，与输入规模无关。
  - 2、对数函数 logn：复杂度与输入量之间是对数关系（对数的底为2）。
  - 3、线性函数 n：复杂度随输入量的增加而线性增加。
  - 4、n*logn：复杂度是线性函数与对数函数的乘积。
  - 5、幂函数 n^c（c是常数）：复杂度是多项式函数。
  - 6、指数函数 a^n（a是常数）：复杂度是指数函数。
  - 7、阶乘 n!： 这类复杂度比指数函数还高。
- 常见复杂度类型的大小次序如下
  - $O(1)<O(\log n)<O(n)<O(n \log n)<O(n^2)<O(n^3)<O(2^n)<O(3^n)<O(n!)<O(n^n)$

基本复杂度类型曲线



基本复杂度类型说明

类 型	名 称	说明
1	常量	为数较少的效率最高的算法。
logn	对数	算法每次循环都会消去问题规模的一个常数因子。
n	线性	扫描规模为n的列表（如顺序查找）算法。
nlogn	n-logn	诸多分治算法，包括合并排序和快速排序平均效率属于此类型。
n^2	二次	一般来说，包含两重嵌套循环的算法。
n^3	三次	一般来说，包含三重嵌套循环的算法。
2^n	指数	求n个元素集合的所有子集的算法。“指数”这个术语常用在更广泛的层面上，不仅包括这种类型，还包括那些增长速度更快的算法如阶乘。
n!	阶乘	求n个元素集合的完全排列的算法。

对数函数的复杂性

- 算法分析中的对数函数 logn，一般是指以 2 为底的对数$\log_2 n$。
- 实际从复杂性角度看，不同底的对数函数其实是同阶的，即：

$$\log_k n = \Theta(\log_l n)$$

此外，对数函数还有两个性质（注意：第一个o是小写o）：

$\log_b n = o(n^\alpha) \quad \alpha > 0$ ，即对数函数的阶小于多项式函数

$a^{\log_b n} = n^{\log_b a}$ ，即如果指数上是对数，实际是多项式函数

2.7 复杂性函数的大类

- 可以将算法复杂性进一步划分为多项式时间复杂性和指数时间复杂性。
- 多项式时间的算法
  - 指时间复杂度函数为 O(p(n)) 的算法，其中 p(n) 是 n 的多项式（即n^c，c为常数）。
  - 多项式时间算法的运行时间增长随问题规模 n 的增长相对较慢，称为“有效算法”。
- 指数时间（不是多项式时间）的算法
  - 指不存在多项式 p(n) 使得该算法的时间复杂度为 O(p(n))，包含指数时间甚至更高阶（如 n! ）的算法。
  - 指数时间（或更高时间）算法当 n 较大时通常是无法应用的，因为运算时间过长。

多项式时间和指数时间对比

	多项式函数						指数函数		
T(n)	logn	n	nlogn	n^2	n^3	n^5	2^n	3^n	n!
n=10	3.3 微秒	10	33	100 微秒	1 毫秒	0.1 秒	1 毫秒	59 毫秒	3.6 秒
n=40	5.3	40	213	1600 微秒	64 毫秒	1.7 分	12.7 天	3855 世纪	10^{37} 世纪
n=60	5.9	60	354	3600 微秒	216 毫秒	1.3 分	366 世纪	10^{13} 世纪	10^{66} 世纪

注：假设每“微秒”执行一步基本操作

多项式时间和指数时间对比（续）

时间复杂度函数	1小时可解的问题实例的最大规模		
	计算机	快100倍的计算机	快1000倍的计算机
n	N_1	$100\ N_1$	$1000\ N_1$
n^2	N_2	$10\ N_2$	$31.6\ N_2$
n^3	N_3	$4.64\ N_3$	$10\ N_3$
n^5	N_4	$2.5\ N_4$	$3.98\ N_4$
2^n	N_5	$N_5 + 6.64$	$N_5 + 9.97$
3^n	N_6	$N_6 + 4.19$	$N_6 + 6.29$

多项式

指数

结论：随着算法复杂性的增加，提高计算机性能带来的收益越来越小。

多项式时间和指数时间对比（续）

- 一台10 亿次/ 秒机器求解不同问题对比
  - 快速排序算法（O(n*logn)）：给 10 万个数据排序，运算量约为$10^5 \log 10^5 \approx 1.7\times 10^6$，仅需$1.7\times 10^6 / 10^9 = 1.7\times 10^{-3}$ 秒.
  - Dijkstra 算法（$O(n^2)$）：求解 1 万个顶点的图的单源最短路径问题，运算量约为$(10^4)^2 = 10^8$，约需$10^8 / 10^9 = 0.1$ 秒。
  - 回溯法（$O(2^n)$）：解 100 个顶点的图的最大团问题, 运算量为$100\times 2^{100} \approx 1.8\times 10^{32}$，需要$1.8\times \times 10^{32} / 10^9 = 1.8\times 10^{21}$秒 = 5.7×10^{15} 年年, 即5千7百万亿年。



三、非递归算法复杂性分析

- 3.1 非递归算法复杂性分析的步骤
- 3.2 常数阶算法
- 3.3 平方阶算法
- 3.4 立方阶算法
- 3.5 常用的求和式
- 3.6 顺序检索算法
- 3.7 矩阵乘法
- 3.8 矩阵元素唯一性验证
- 3.9 欧几里得算法

3.1 非递归算法复杂性分析的步骤

- 1、确定用哪个（些）参数作为输入规模的度量参数
  - 排序：数组中元素个数n
  - 检索：被检索数组的元素个数n
  - 整数乘法：两个整数的位数 m，n
  - 矩阵相乘：矩阵的行列数 i，j，k
  - 图的遍历：图的顶点数n，边数m
  - ……
- 2、确定算法基本操作
  - 一般情况下是算法最内层循环里最耗时的操作；
  - 常见基本操作：比较、加法、乘法、置指针、交换、……

非递归算法复杂性分析的步骤（续）

- 3、考虑基本操作的执行次数是否仅仅与输入规模有关
  - 如果是，则直接求解；
  - 若还与其他因素有关，则按需要对最差、最优、平均效率作出分析。
- 4、建立基本操作执行次数与算法输入规模的求和表达式。
- 5、利用求和运算公式和法则求解算法基本操作的执行次数，确定算法的复杂度类型。

3.2 常数阶算法

算法：交换两个变量的值

```
void Swapij(int * i, int * j)
{
    int temp;
    temp = &i;
    &i = &j;
    &j = temp;
}
```

分析过程：

1、输入规模：两个变量

2、基本操作：赋值

3、基本操作只和输入规模有关

4、执行次数：C(n)=3

5、算法复杂度：O(1)

结论：如果算法的执行时间不随着问题规模 n 的增加而增长，即使算法中有上千条语句，其执行时间也不过是一个较大的常数。此类算法的时间复杂度是O(1)，即常数阶。

3.3 平方阶算法

算法：累加函数

```
void calculate1(int n)
{
    int x = 0;
    int y = 0;
    for(int k = 1; k <= n; k++)
        x += 1;
    for(int i = 1; i <= n; i++)
        for(int j = 1; j <= n; j++)
            y += 1;
}
```

分析过程：

1、输入规模：整数 n

2、基本操作：加法

3、基本操作只和 n 有关

4、执行次数：$C(n)=n+n^2$

5、算法复杂度：$O(n^2)$

$$C(n)=\sum_{k=1}^{n}1+\sum_{i=1}^{n}\sum_{j=1}^{n}1=n+n^2$$

结论：当有若干个循环语句时，算法的时间复杂度是由嵌套层数最多的循环语句中的最内层操作（关键操作）决定，基本操作要从最内层循环的操作中选择一个最耗时的操作。

3.4 立方阶算法

算法：累加函数2

```
void calculate2(int n)
{
    int x = 1;
    for(int i = 1; i <= n; i++)
        for(int j = 1; j <= i; j++)
            for(int k = 1; k <= j; k++)
                x += 1;
}
```

注意每层的循环次数受上层控制

注意每层的循环次数受上层控制

可以不考虑循环本身的加法、比较操作，不影响算法的复杂性

分析过程：

1、输入规模：整数 n

2、基本操作：加法

3、基本操作只和 n 有关

4、执行次数： $C(n)=\frac{n^3+3n^2+2n}{6}$

5、算法复杂度：$O(n^3)$

未考虑循环本身的加法等操作

立方阶算法（续）

C(n)的计算过程（注意求和的上下界）：

$$\sum_{i=1}^{n}\sum_{j=1}^{i}\sum_{k=1}^{j}1=\sum_{i=1}^{n}\sum_{j=1}^{i}j=\sum_{i=1}^{n}i(i+1)/2=[n(n+1)(2n+1)/6+n(n+1)/2]/2$$

$$=\frac{n^3+3n^2+2n}{6}$$

注意：计算中用到了常用的求和式，如下：

$$1+2+3+\ldots\ldots+n=\frac{n(n+1)}{2}$$

$$1+2^2+3^2+4^2+\ldots\ldots+n^2=\frac{1}{6}n(n+1)(2n+1)$$

$$1+2^3+3^3+4^3+\ldots\ldots+n^3=\left[\frac{1}{2}n(n+1)\right]^2$$

$$1+3+5+\ldots+(2n-1)=n^2$$

3.5 常用的求和式

$$\sum_{k=1}^{n} a_k = \frac{n(a_1 + a_n)}{2}$$ 等差数列求和

$$\sum_{k=0}^{n} aq^k = \frac{a(1-q^{n+1})}{1-q}$$

$$\sum_{k=0}^{n} x^k = \frac{1-x^{n+1}}{1-x}$$

} 等比数列求和

$$\sum_{k=1}^{n} \frac{1}{k} = \ln n + O(1)$$ 调和级数

3.6 顺序检索算法

```
int SeqSearch(A[0..n-1], K)
{
    i = 0;
    while ((i<n) && (A[i]<>K))
        i++;
    if (i == n) i = -1;
    return i;
}
```

分析过程：

1、输入规模：数组元素个数 n

2、基本操作：加法（比较）

3、基本操作不光和 n 有关，还和键值 K 有关

4、执行次数：分为最差、最优和平均

5、平均复杂度：O(n)

$C_{best}(n) = 1$，即最优情况下只需比较1次。

$C_{worst}(n) = n$，即最差情况下需比较n次。

$C_{avg}(n) = \sum_I p(I)C(I) = n(1-p) + p(n-1)/2$

3.7 矩阵乘法

```
//输入是两个二维方阵,输出是二者的乘积,方阵是 n 阶的
void CMatrixMulti(A[1..n][1..n], B[1..n][1..n], C[1..n][1..n])
{
    for(int i = 1; i <= n; i++)
        for(int j = 1; j <= n; j++)
        {
            C[i][j] = 0;
            for(int k = 1; k <= n; k++)
                C[i][j] += A[i][k] * B[k][j];
        }
    }
```

分析过程：

1、输入规模：矩阵阶数 n

2、基本操作：乘法（乘法比加法更加耗时）

3、基本操作只和 n 有关

4、执行次数：$C(n)=\sum_{i=1}^{n}\sum_{j=1}^{n}\sum_{k=1}^{n}1=n^3$

5、算法复杂度：$O(n^3)$

3.8 矩阵元素唯一性验证

```
//输入已经转换成一维数据的形式
//输出如果没有两个元素相同,则为 true; 否则为 false
bool CalUniqueMatrix(A[1..n])
{
    for(int i = 1; i <= n - 1; i++)
        for(int j = i + 1; j <= n; j++)
            if(A[i] == A[j]) return false;
    return true;
}
```

注意每层的循环次数受上层控制

分析过程：

1、输入规模：数组元素个数 n

2、基本操作：比较

3、基本操作不光和 n 有关，还和数组A的值有关

4、执行次数：分为最差、最优和平均

5、算法复杂度：$O(n^2)$

矩阵元素唯一性验证（续）

- 最优效率：$C_{best} = 1$
- 平均效率：
  - 受矩阵元素概率分布的影响，比较复杂。
- 最差效率：

$$C_{worst} = \sum_{i=1}^{n-1} \sum_{j=i+1}^{n} 1 = \sum_{i=1}^{n-1} (n-i)$$

$$= \sum_{i=1}^{n-1} n - \sum_{i=1}^{n-1} i = (n-1)n - \frac{(n-1)n}{2}$$

$$= \frac{(n-1)n}{2} \in O(n^2)$$

3.9 欧几里得算法

算法：GCD(m,n)
输入：两个正整数 m 和 n
输出：m和n的最大公约数
伪代码：

```
1. r ← m mod n
2. while r<>0 do
3.     m ←n
4.     n ←r
5.     r ←m mod n
6. return n
```

- 有时候算法时间的计算并非直截了当，while循环每次迭代后余数值并非按常数因子递减。
  - 如何计算迭代次数？
- 一个定理：
  如果 n>m ，
  则 n mod m < n/2
- 每次迭代的余数最多是原始值的一半，因此迭代次数为2O(logn)。
- 因此欧几里德算法的时间复杂度为 O(logn)。

四、递归算法复杂性分析

- 4.1 递归算法复杂性分析的步骤
- 4.2 求 n 的阶乘 n!
- 4.3 汉诺塔问题

4.1 递归算法复杂性分析的步骤

- 1、决定用哪个（或哪些）参数作为输人规模的度量。
- 2、找出算法的基本操作
  - 递归算法的基本操作一般是在递归调用附近最耗时的操作。
- 3、确定基本操作的执行次数是否只依赖于输人规模
  - 若是则直接求解；
  - 否则需要考察算法的最差、最优及平均效率。
- 4、建立算法中基本操作执行次数的“递推关系式”及其“初始条件”。
- 5、求解递推关系式以确定它的复杂性类型。

4.2 求 n 的阶乘 n!

迭代算法

```
int fact1(int n)
{
    int fact = 1;
    for(int i = 1; i <= n; i++)
        fact = fact * i;
    return fact;
}
```

递归算法

```
int fact2(int n)
{
    if (n == 0 || n == 1) return 1;
    else return (n * fact2(n-1));
}
```

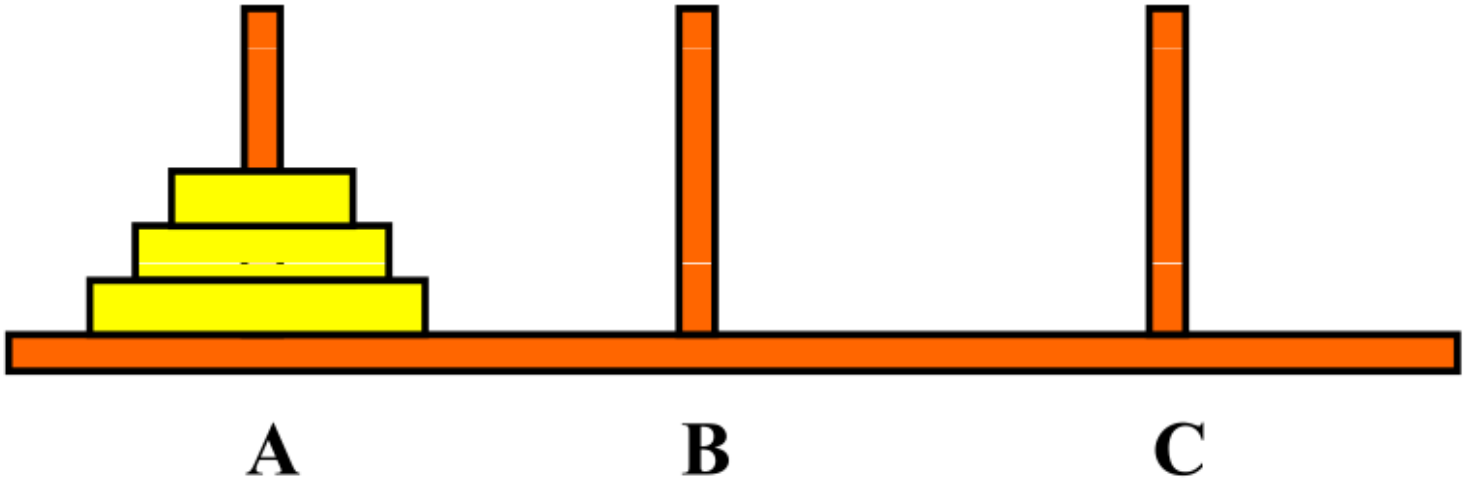
每次递归时做一次乘法

1、输入规模：整数 n
2、基本操作：乘法
3、基本操作只和 n 有关
4、递推关系式：T(n)=T(n-1)+1，T(1)=0（每次递归1 次乘法）
5、解递推关系式：T(n)=T(n-1)+1=T(n-2)+1+1=T(n-2)+2=
T(n-3)+3=…=T(n-(n-1))+n-1=n-1
6、算法复杂度为 O(n)

递推到初始条件

4.3 汉诺塔问题

- 有三根相邻的柱子，标号为 A、B、C。
- A 柱子上从下到上按金字塔状叠放着 n 个不同大小的圆盘，现在要把所有盘子一个一个地移动到柱子C上，并且每次移动同一根柱子上都不能出现大盘子在小盘子上方的情况，请问至少需要多少次移动。



汉诺塔问题（续）

- 使用递归算法求解问题，考虑 n 个盘子的移动
  - 先将前 n-1个盘子从 A 柱移到 B 柱，再把第 n 个 盘子从 A 柱移到 C 柱，再将前 n-1 个盘子从 B 柱移到C 柱。
  - 前 n-1 个盘子如何从 A 柱移到 B 柱与原问题的求解思路相同，这就是递归算法的思想。

```
void Hanoi(int n, A, B, C)
{
    if(n == 1) MoveOne(n, A, C);
    else
    {
        Hanoi(n - 1, A, C, B);
        MoveOne(n, A, C);
        Hanoi (n - 1, B, A, C);
    }
}
```

当问题规模为1 时，可以直接解决

递归调用规模为 n-1 的问题

MoveOne为移动一个盘子的算法，常数阶

汉诺塔问题（续）

1、输入规模：整数 n

2、基本操作：MoveOne

3、基本操作只和 n 有关

4、递推关系式：

$T(n)=2*T(n-1)+1$，

$T(1)=1$

5、解递推关系式：

$T(n)=\ldots\ldots=2^n-1$（推导过程见后）

6、算法复杂度为 $O(2^n)$



五、递归方程的求解方法

- 5.1 直接迭代
- 5.2 换元迭代
- 5.3 递归树
- 5.4 主定理

5.1 直接迭代

- 汉诺塔问题

$$T(n)=2*T(n-1)+1，T(1)=1$$

- 直接迭代

$T(n)= 2*T(n-1)+1$

$=2*(2*T(n-2)+1)+1 = 2^2*T(n-2)+2+1$

$=4*(2*T(n-3)+1)+2+1 = 2^3*T(n-3)+2^2+2+1$

=…（找规律）

$=2^{n-1}*T(n-(n-1))+2^{n-2}+…+2+1$

$=2^{n-1}+2^{n-2}+…+2+1$

$=2^n-1 \in O(2^n)$

递推到初值 T(1) 为止

等比数列

5.2 换元迭代

- 有时对n直接迭代不方便，可以换元迭代。
- 基本过程
  - 1、将对 n 的递推式换成对其他变量 k 的递推式。
  - 2、对 k 进行直接迭代。
  - 3、将迭代结果（关于 k 的函数）转换回关于 n 的函数。
- 例：递推方程如下（假设 $n= 2^k$ ）：

$$T(n)=2T(n/2)+n$$

$$T(1)=0$$

求解该递推表达式。

换元迭代（续）

- 将 n 换成 k，进行迭代（$n = 2^k$）

$$T(n)=T(2^k)=2T(2^{k-1})+2^k$$
$$=2(\ 2T(2^{k-2})+2^{k-1})\ +\ 2^k$$
$$=2^2\ T(2^{k-2})+2^k+2^k$$
$$=2^2\ (2T(2^{k-3})+2^{k-2})\ +\ 2^k\ +\ 2^k$$
$$=2^3\ T(2^{k-3})+3\ *\ 2^k$$

……（找规律，递推到初值）

$$=2^kT(2^{k-k})\ +\ k\ *\ 2^k$$
$$=2^kT(1)\ +\ k\ *\ 2^k$$
$$=\ 0\ +\ k\ *\ 2^k$$
$$=\ n\ *\ \log n \in O(n*\log n)$$

5.3 递归树

- 递归方程可以采用递归树求解
  - 递归树也是一种迭代计算模型，其生成过程和直接迭代的过程一致。
  - 递归树中的所有项恰好是直接迭代之后产生的和式中的项。
  - 递归方程的解就是递归树上“所有项”之和。
  - 递归树以可视化方式生成，在求解递归方程时更加直观。

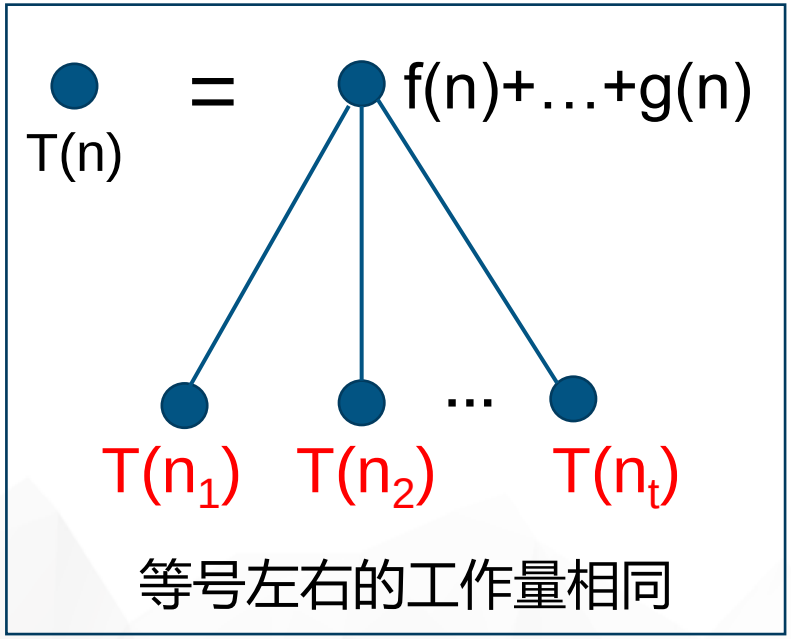
递归树的基本原理

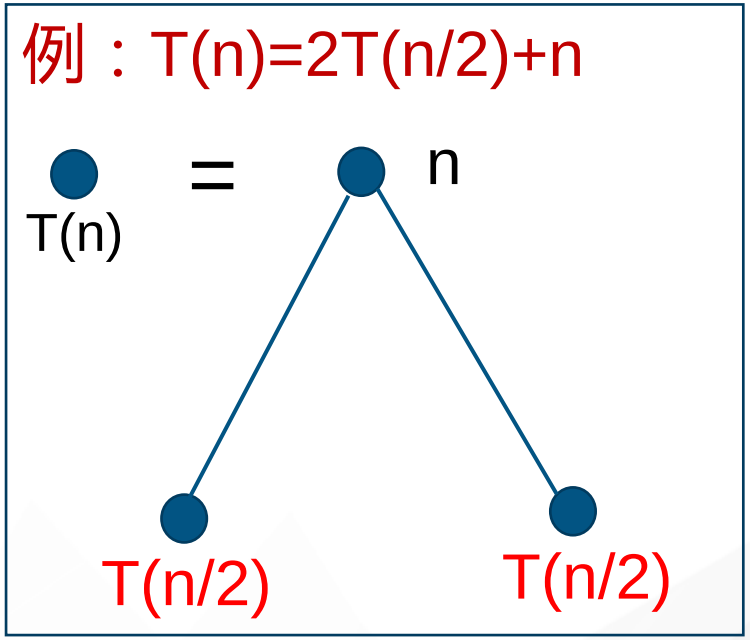
- 如果递归树上某节点记为 T(n)，且

  $T(n)=T(n_1)+T(n_2)+\ldots+T(n_t)+f(n)+\ldots+g(n)$，

  $(n_1、n_2、\ldots\ldots、n_t <= n)$

  则可以把 T(n) 表示为两层节点之和：



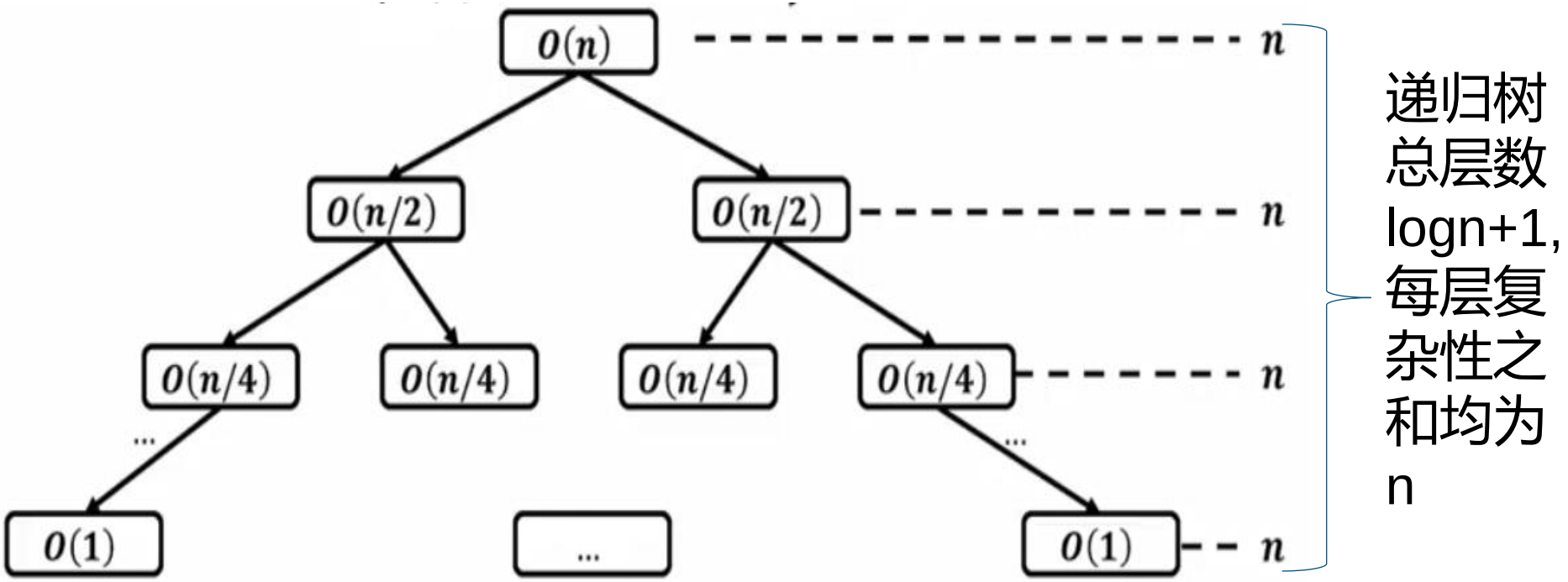


递归树的生成步骤

- 由递归树的基本原理可得到生成步骤：
  - 1、初始时，递归树只有根节点，其值为T(n)。
  - 2、不断继续下述过程，直到递归树的叶节点都是递归的“初值”为止：
    - （1）将迭代树的叶节点展开成工作量相同的二层子树。
    - （2）用二层子树替换该叶节点。

递归树例1

- T(n)=2T(n/2)+O(n)，T(1)=1（设$n=2^k$）
- 最终生成的递归树如下（注意掌握其生成的动态过程）

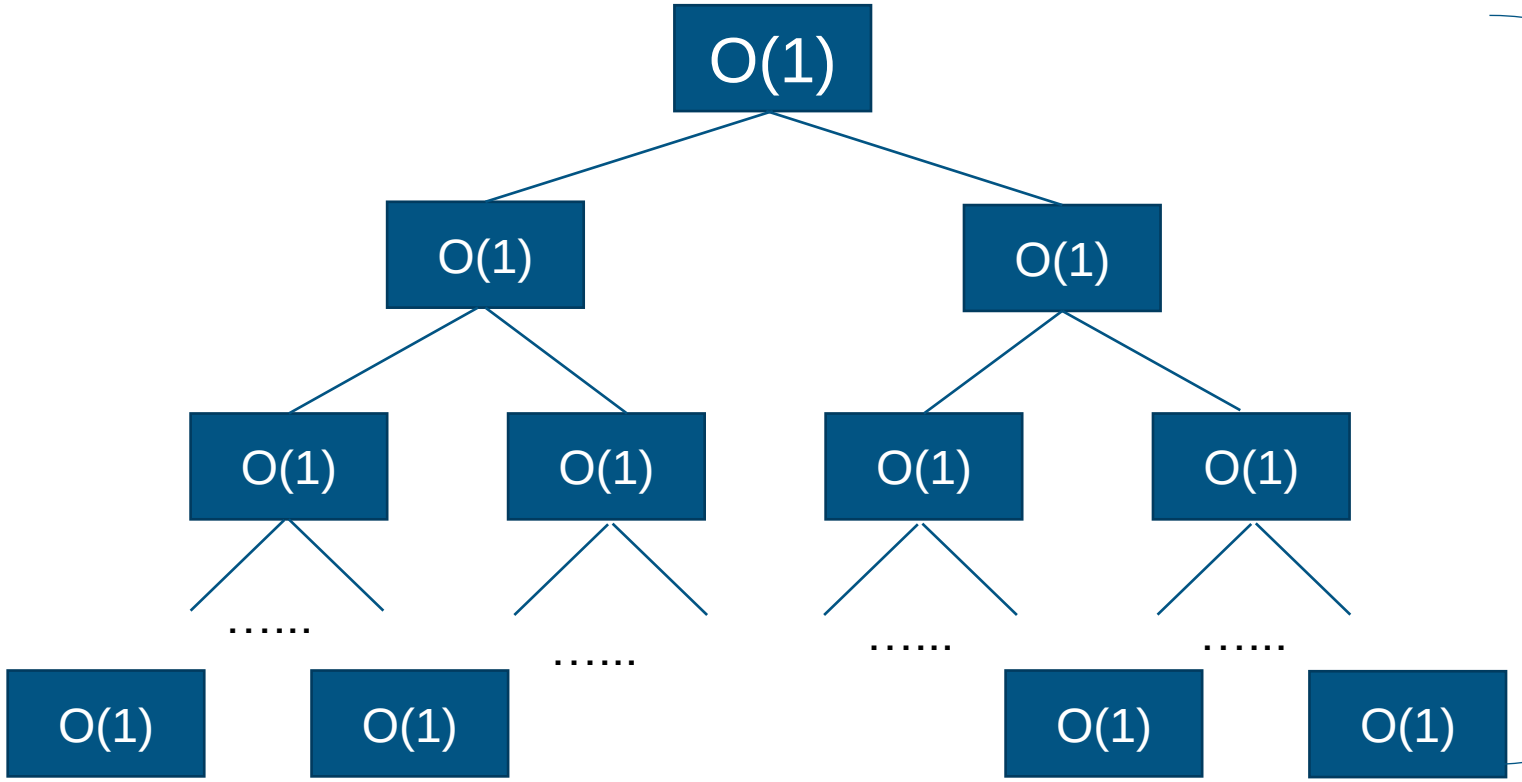


递归树总层数 logn+1，每层复杂性之和都是 n，所以：

$T(n) = (\log n+1)*n = n*\log n+n \in O(n*\log n)$

递归树例2

- $T(n)=2*T(n-1)+O(1)$，$T(1)=1$

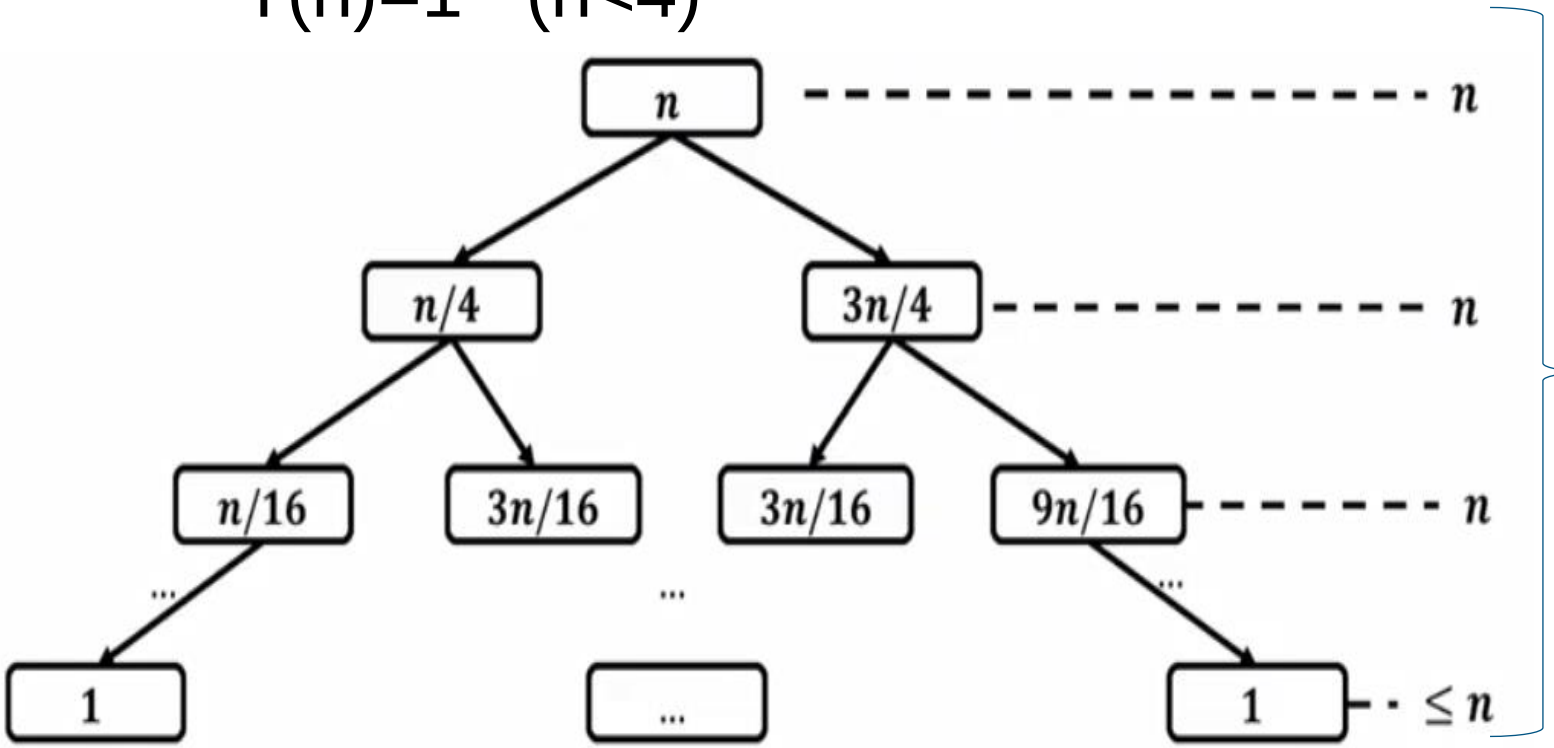


递归树总层数为 n，每层工作量之和为 1、2、4、…，2^{n-1}

所以 $T(n)=1+2+2^2+\ldots+2^{n-1}= 2^n-1 \in O(2^n)$

递归树例3

- T(n)=T(n/4)+T(3n/4)+n　(n>=4),
  T(n)=1　(n<4)



递归树层数最多为：$\log_{\frac{4}{3}} n$

所以 T(n) < n* $\log_{\frac{4}{3}} n$ ∈ O(n*logn)

5.4 主定理

- 主定理提供了求解如下形式递归方程的一般方法：

$$T(n)=a*T(n/b)+f(n)$$

其中$a\geq 1$，$b>1$为常数

- 该递归方程将规模为 n 的问题分解成 a 个子问题，每个子问题的大小为 n/b。
- 例如，对于递归方程

$$T(n)=3T(n/4)+n^2，$$

有：$a=3$，$b=4$，$f(n)=n^2$。

主定理（续）

- 主定理：
  - 对形如 T(n)=a*T(n/b)+f(n) （其中a ≥ 1，b>1 的常数）的递归方程，有以下结论：

1. 若$f(n)=O(n^{\log_b a-\varepsilon}),\varepsilon>0$，那么$T(n)=\Theta(n^{\log_b a})$
2. 若$f(n)=\Theta(n^{\log_b a})$，那么$T(n)=\Theta(n^{\log_b a}\log n)$
3. 若$f(n)=\Omega(n^{\log_b a+\varepsilon}),\varepsilon>0$，且对于某个常数$c<1$和充分大的$n$有 $af(n/b)\le cf(n)$，那么$T(n)=\Theta(f(n))$

主定理实际是把 f(n) 与 $n^{\log_b a}$ 进行比较，根据二者的大小关系直接得到递归方程 T(n) 的解。

主定理（续）

- f(n) 与$n^{\log_b a}$的大小与递归方程的解：
  - 情况1：函数 f(n) 比函数 $n^{\log_b a}$更小，则T(n)=O($n^{\log_b a}$) 。
  - 情况2：函数 f(n) 和 函数 $n^{\log_b a}$ 一样大，则 T(n)=O($n^{\log_b a}\log_2 n$)。
  - 情况3：函数 f(n) 比函数 $n^{\log_b a}$ 更大，则T(n)=O(f(n))。

主定理（续）

- 主定理的简化形式
  - 对形如 $T(n)=a*T(n/b)+n^k$（其中$a \geq 1$，$b>1$ 为常数）的递归方程，可以应用主定理简化形式，有以下结论：

$$T(n)=\begin{cases}\Theta(n^k) & if\ k>\log_b a\\ \Theta(n^k \log n) & if\ k=\log_b a\\ \Theta(n^{\log_b a}) & if\ k<\log_b a\end{cases}$$

主定理（续）

- 例1：分析以下递归方程的时间复杂度

  $T(n)=4T(n/2)+n$ 当$n>1$

  $T(n)=1$ 当$n=1$

  解： a =4， b=2，$\log_b a$= 2， f(n)=n 。

  $f(n)=n^1$，即 k=1（ f(n)比n^2小 ）。

  所以 $T(n)=O(n^{\log_b a})=O(n^2)$

- 例2：分析以下递归方程的时间复杂度

  $T(n)=2T(n/2)+n^2$，当n>1； T(n)=1，当n=1

  解： a =2， b=2，$\log_b a$= 1， $f(n)=n^2$ 。

  $f(n)=n^2$，即 k=2（ f(n) 比 n^1大 ）。

  所以 $T(n)=O(f(n))=O(n^2)$

本章小结

- 一、算法分析框架
- 二、算法渐进复杂性分析
- 三、非递归算法复杂性分析
- 四、递归算法复杂性分析
- 五、递归方程的求解方法